

HelixSkills

HelixSkills

CLI AI نظام مهارات مدعوم بحوكمة ودستور لموفري

ملخص

كوحدة فرعية Helix Constitution يرث CLI AI هو نظام مهارات لموفري HelixSkills وبالتالي تُطبق جميع قواعد الحوكمة العالمية دون استثناء. يجمع هذا النظام مهارات الموفرين القابلة والمحركات القابلة لإعادة، Claude، ومكونات الإضافات البرمجية، MCP، للتثبيت، وخوادم الأنواع. الاستخدام خلف كتالوج قابل للتسجيل وموثق.

وصف مختصر

كوحدة فرعية Helix Constitution يدمج CLI AI هو نظام مهارات لموفري HelixSkills بحيث تُطبق جميع القواعد العالمية، ثم يقدم مهارات قابلة للتسجيل (نظام بادئة الإجراءات، مدقق الوسائط، المسار المتعدد، مزامنة الجلسة، دورة حياة العناصر القابلة للعمل، وغيرها)، وخادمي أدوات ومحركات قابلة لإعادة الاستخدام، Claude، ومكوني إضافات برمجية، MCP.

وصف مفصل

CLI AI هو نظام مهارات لموفري (Apache-2.0 ترخيص، skills مستودع) HelixSkills كوحدة Helix Constitution ويبدأ من قلب متعمد للترتيب المعتاد: الحوكمة أولاً، ثم القمرة. يرث constitution/ وبالتالي تُطبق جميع القواعد العالمية من، constitution/، فرعية CLAUDE.md وconstitution/Constitution.md ليس كعرف قد يحترمه. الموفر، بل كمجموعة قواعد مدمجة فعلياً في هيكل المشروع. لا يمكن للموفر الذي يعتمد أن يتجاهل الدستور؛ فالقواعد تنتقل مع الكود HelixSkills.

مخزوناً ملموساً وقابلاً HelixSkills بينما تتعامل معظم “أطر المهارات” مع المفاهيم المجردة، يقدم نظام بادئة register.sh للتسجيل يمكنك الإشارة إليه وتثبيته. تُثبت سبع مهارات دستورية عبر الإجراءات، مدقق الوسائط، المسار المتعدد، إعدادات تقارير العناصر القابلة للعمل، قائمة العمل المجدولة مزامنة الجلسة، ودورة حياة العنصر القابل للعمل — وهي مجموعة تتراوح من المستوى المتوسط إلى المتقدم، تغطي كل شيء بدءاً من تسمية الإجراءات المنظمة وصولاً إلى التحقق من الوسائط ودورة حياة Java/وحدة العمل الكاملة. كما تم فهرسة مهارات تجريبية إضافية (نظرة عامة على أندرويد، لغة مدقق الوسائط، والعمل) MCP وجاهزة للتنشيط. يقدم خادماً أدوات (Linux نظام التشغيل، Kotlin،

بينما تضيف مكوناً Model Context Protocol (المجدول) هذه المهارات للموفرين عبر نفس القدرات مباشرة إلى بيئة تشغيل (والعمل المجدول، Claude (helix الإضافات البرمجية الموفر — مجموعة مهارات واحدة تصل إلى الموفرين عبر أي واجهة يتواصلون من خلالها.

continuum — تحت الكتلوج توجد أربع محركات قابلة لإعادة الاستخدام من المستوى الأول session_orchestrator، token_optimizer، وclickup_sync بالإضافة إلى، (مُنَفَّذ) وهي الآليات المشتركة التي تمنع المهارات من إعادة اختراع نفس البنية التحتية — (قيد التصميم) vasic-digital يعلن مُحسّن الرموز وحده عن رسم بياني واضح للتبعيات يمتد إلى حزم منظومة (TOON، Embeddings، VectorDB، Normalize، conversation) مما يجعل توصيلاته عبر المستودعات، HelixDevelopment الخاص بـ LLMProvider و قابلة للتدقيق بدلاً من أن تكون ضمنية. يحيط بكل ذلك توثيق منظم: كتالوج المهارات، فهرس مهارات مُؤَدَّ آلياً، صفحات تفصيلية لكل مستودع، وسجل صريح للفجوات والمخاطر يحدد ما لم يُنجز بعد لضمان المرونة والوصول GitVerse، وGitFlic، وGitLab، وGitHub، يُعكس النظام بأكمله عبر الإقليمي.

المحتوى

لماذا قمنا ببنائه

إلى قرارات متسقة ومحكومة وقابلة لإعادة الاستخدام، لا إلى نصوص برمجية AI و CLI يحتاج وكلاء لمنح الوكلاء مجموعة مهارات مجمعة HelixSkills مؤقتة تعيد ابتكار القواعد في كل مرة. بُني وقابلة للتسجيل، مرتبطة بدستور مشترك، بحيث يظل السلوك متسقاً وقابلاً للتدقيق عبر كل وكيل ومشروع يعتمد عليه.

لماذا يُعد نقطة تحول

لا عبر الانضباط فقط. فكل مهارة هي وحدة، بشكل بنيوي يجعل قرة الوكلاء محمولة ومتوافقة مع القواعد محكومة ومُمرّجة بإصدار محدد وقابلة للتنشيط، مدعومة بوحدة فرعية دستورية—ففي اللحظة التي يسجل فيها الوكيل مهارة ما، يرث تلقائياً مجموعة القواعد المرجعية دون أي مجال للانحراف. وهذا يفتح الباب أمام ما لم يكن ممكناً من قبل: نقل قرة من وكيل أو مشروع إلى آخر مع العلم بأنها تصل مرتبطة بدلاً (Claude Code وملحقات MCP خوادم) بالفعل بنفس الحوكمة، وتُعرض عبر واجهات قياسية من مجموعة نصوص برمجية مخصصة تعيد ابتكار القواعد في كل مرة.

ما الجديد في هذا النهج

- تُورث قواعد الحوكمة العالمية بدلاً من نسخها—تُدمج في: كوحدة فرعية **Constitution** الهيكل بحيث يرتبط كل وكيل مستهلك بنفس مجموعة القواعد المرجعية، مع تحديثات تصدر من مصدر واحد للحقيقة بدلاً من عشرات النسخ القديمة.
- وتُنسج في فهرس رسم بياني للمهارات (register.sh) المهارات تُقدّم كوحدة ذاتية التسجيل. يُؤَدَّ تلقائياً، بحيث يظل الكتلوج قابلاً للاكتشاف ولا ينفصل أبداً عما هو مثبت فعلياً.
- MCP التعرض عبر واجهات متعددة: تصل مجموعة المهارات ذاتها إلى الوكلاء عبر خوادم أدوات. اكتب مرة واحدة، وتحدث مع أي بيئة تشغيل يستخدمها الوكيل—Claude Code ملحقات و.
- (continuum، token_optimizer، session_orchestrator، clickup_sync) محركات قابلة لإعادة الاستخدام من المستوى الأول مشتركة عبر النظام البيئي، تحمل كل منها (session_orchestrator، clickup_sync). إعلانات تبعية صريحة وقابلة للتدقيق عبر المستودعات بدلاً من الارتباطات المخفية.

أكبر التحديات التقنية وكيف تغلبنا عليها

- إعادة—الحفاظ على سلوك الوكلاء متسقاً ومتوافقاً مع القواعد عبر مهارات ووكلاء متعددين Helix تنفيذ الحوكمة لكل مهلة يضمن التباعد مع مرور الوقت. تم حل هذه المشكلة بدمج constitution/CLAUDE.md كوحدة فرعية بحيث تُطبق القواعد في constitution/Constitution.md دون شروط وتُحدّث من مصدر واحد بدلاً من نسخها وتركها لتتغفن.
- فهرس المهارات لا قيمة له إذا لم—جعل مجموعة المهارات المتنامية قابلة للتثبيت والاكتشاف يتمكن أحد من العثور على ما فيه أو تثبيته. تم حل هذه المشكلة من خلال تسجيل كل مهلة عبر عند التثبيت، بالإضافة إلى فهرس رسم بياني للمهارات يُولّد تلقائياً ووثائق register.sh تفصيلية لكل مستودع، بحيث يتتبع الاكتشاف الواقع تلقائياً.
- لا ينبغي إعادة بناء نفس القمرة لكل مضيف—الوصول إلى وكلاء يتحدّثون بيانات تشغيل مختلفة تحت MCP) تم حل هذه المشكلة بتغليف مجموعة المهارات ذاتها خلف تعريفات خادوم أدوات constitution/mcp/) Claude Code وملحقات constitution/plugins/). بحيث تُعرض تنفيذية واحدة عبر واجهات متعددة.

المحتوى

مجموعة الأدوات التقنية

- اختيرت لأنها تسمح لأدوات التثبيت والتسجيل بالعمل في أي — (اللغة الأساسية) **Shell** مكان يتواجد فيه العميل، دون الحاجة إلى تحميل بيئة تشغيل مسبقة؛ فهي تشغل مما يحافظ على مدخل خالٍ من التبعيات وسهل، install_upstreams و register.sh النقل.
- Helix اختيرت لورثة الحوكمة دون تكرار: حيث يتم تحميل — الفرعية **Git** وحدات كمرجع حي، بحيث تنتشر تحديثات القواعد constitution/ في المسار Constitution عبر مؤشر واحد بدلاً من نسخها ولصقها ثم نسيانها.
- اختيرت كواجهة معيارية للأدوات مستقلة — **Model Context Protocol (MCP)** MCP (media-validator عن بيئة التشغيل للعملاء؛ حيث تم تعريف خادمي لعرض المهارات كأدوات قابلة constitution/mcp/ تحت المسار scheduled-work) للاستدعاء.
- اختيرت لإدماج المهارات بشكل أصلي في بيئة تشغيل العميل — البرمجية **Claude** ملحقات (helix و scheduled-work) دون الحاجة إلى أي ربط إضافي؛ حيث يتم شحن ملحقيّن لاستضافة مختلفة MCP مما يعكس واجهة، constitution/plugins/ تحت المسار.
- اختيرت لاستخلاص الآليات — (continuum.token_optimizer، session_orchestrator، clickup_sync) المحركات القابلة لإعادة الاستخدام token_optimizer vasic-digital (TOON، Embeddings، VectorDB، Normalize، conversation) و HelixDevelopment's LLMProvider من خلال تعريف التبعيات بدلاً من تكرار الأكواد.
- اختيرت لضمان عدم قطع الوصول في حال تعطل مضيف واحد أو حظر إقليمي؛ حيث يتم الاحتفاظ بنفس المستودع نشطاً عبر أربعة منصات لتحقيق المرونة والوصول الشامل.

ملاحظات الحالة والشفافية

- وملحقين؛ بينما لا تزال MCP تم شحن المهارات السبع للدستور وخدمي .الحالة: تجريبية المهارات المسودة مفهومة بانتظار التنشيط، وثلاثة من المحركات الأربعة ذات العمق الأول قيد التصميم (session_orchestrator, token_optimizer, clickup_sync)
- ؛ بينما المسار الرسمي في helix_skills إلى المشروع باسم README يشير ملف كما أن عدد النتائج المتنبعة في ملف HelixDevelopment/skills هو GitHub هو رقم ذاتي الإبلاغ README

أساسية-Helix: درجة الأولوية